Astheroth™ / ACRE

A Deterministic Admissibility Framework for Physical and Computational Systems

Alexander Wolfgang Strüver
Thailand

March 30, 2026

Abstract

Modern computational and physical systems rely predominantly on probabilistic inference, heuristic optimization, and model-dependent approximation. While effective in many domains, these approaches fundamentally lack structural guarantees regarding correctness, admissibility, and reversibility of decisions—especially at irreversible execution boundaries.

This paper introduces a deterministic, ontology-derived evaluation framework that replaces probabilistic decision-making with strict admissibility constraints. The system enforces that every output must be structurally derived, uniquely determined, and externally falsifiable. Ambiguity is not resolved through approximation but rejected as underdetermined; contradictions are not tolerated but blocked at execution level.

At its core, the framework defines a minimal, closed identity linking system capacity partition and propagation behavior, from which all admissible regimes are derived. This identity enables a single-parameter control of system states while preserving domain closure and ensuring bounded, real-valued operation across all admissible conditions.

The framework is not limited to theoretical constructs. It is directly mappable to real-world systems through deterministic proxy normalization, invariant-bound commit gates, and hardware-compatible execution pipelines. Implementations include a Universal Stability Kernel (USK) for domain-independent admissibility enforcement, a Universal Physical Admissibility and Stabilisation Layer (UPASL) spanning multiple physical domains, and domain-specific realizations such as thermodynamic commit-gating systems.

Crucially, the framework is fully falsifiable. It produces measurable predictions, defines explicit operational test procedures, and enables independent validation without requiring disclosure of internal system structure. Bidirectional applicability allows both forward prediction and reverse diagnostic reconstruction of system stability states across domains.

This work does not propose another model for approximating reality. It defines a deterministic substrate for evaluating whether actions are admissible within reality-bound constraints. As such, it establishes a new class of compute and control architecture where decision equals proof.

Contents

1. Problem Definition

1.1 Structural Limitations of Current Systems

Contemporary systems—whether in artificial intelligence, control engineering, or physical modeling—share a common structural limitation:

They generate outputs without guaranteeing that those outputs are **admissible**.

This manifests in several fundamental issues:

- **Probabilistic ambiguity**
  Systems produce results based on likelihood rather than necessity.
- **Heuristic decision-making**
  Outputs depend on optimization strategies rather than structural validity.
- **Lack of falsifiability**
  Many results cannot be independently verified or refuted under controlled conditions.
- **Execution without admissibility guarantees**
  Actions may be performed despite insufficient evidence or violated constraints.
- **Drift under adaptation**
  Systems change behavior over time without preserving invariant correctness.

These limitations are not implementation errors. They are structural properties of probabilistic and heuristic frameworks.

1.2 The Irreversibility Problem

The problem becomes critical at **irreversible execution boundaries**, where actions cannot be undone within the operational horizon.

Examples include:

- high-power system activation (e.g. transmission ramp-up)
- mechanical deployment or structural actuation
- financial or transactional commitments
- autonomous system decisions in safety-critical environments

In such cases, incorrect decisions are not merely suboptimal—they are **physically or economically irreversible**.

Current systems attempt to manage this risk through:

- redundancy

- safety margins
- statistical confidence thresholds

However, these approaches do not solve the core issue:

They do not guarantee that a decision is **admissible**.

1.3 Absence of Deterministic Admissibility

A structurally admissible system must satisfy three conditions:

1. **Completeness of evidence**
   Decisions must only be made when sufficient, valid data is available.
2. **Invariant compliance**
   All system constraints must be satisfied at the moment of decision.
3. **Deterministic evaluation**
   The same input must always produce the same output.

Existing systems fail to enforce these conditions simultaneously.

In contrast, the presented framework enforces:

- evidence validity as a prerequisite for evaluation
- invariant-bound decision gates
- fail-closed behavior in case of uncertainty

For example, if required evidence is missing or invalid, the system state is explicitly classified as *undetermined*, and execution is blocked rather than approximated.

1.4 Consequence: Decision Without Proof

The core issue can be summarized as follows:

> Modern systems allow decisions without proof of admissibility.

This leads to:

- hidden failure modes
- non-reproducible behavior
- unverifiable outputs
- uncontrolled risk propagation

In contrast, a deterministic admissibility framework requires:

- every decision to be structurally derivable
- every output to be externally testable
- every execution to be bounded by invariant constraints

1.5 Required Shift

To overcome these limitations, a fundamental shift is required:

From:

- approximation
- probability
- optimization

To:

- admissibility
- determinism
- falsifiability

This paper introduces such a framework.

It does not attempt to improve prediction accuracy. It replaces the decision paradigm entirely.

2. Core Principle

2.1 Minimal Ontological Identity

At the foundation of the framework lies a single closed identity that defines the admissible relationship between bound and propagating system capacity:

$$\left(\frac{v_\Phi}{C_\Phi}\right)^2 + \frac{M}{E} = 1, \quad 0 \leq \frac{M}{E} \leq 1 \tag{1}$$

Where:

- E denotes total transition capacity
- M denotes the bound (stabilized) fraction of that capacity
- v_Φ denotes the effective propagation rate
- C_Φ denotes the maximal admissible propagation rate

The ratio

$$\varepsilon = \frac{M}{E} \tag{2}$$

defines the **regime parameter** of the system.

This identity is not an empirical fit. It is a **closure condition**.

2.2 Forced Consequences (Not Assumptions)

From this identity, several properties follow necessarily.

Domain Closure

$$0 \leq v_\Phi \leq C_\Phi \tag{3}$$

All admissible system states remain within real-valued, bounded domains. No imaginary or undefined regimes exist.

Extreme Regimes

- $\varepsilon = 0 \Rightarrow v_\Phi = C_\Phi$ maximal propagation
- $\varepsilon = 1 \Rightarrow v_\Phi = 0$ complete binding

These are not limits imposed externally. They are **structurally enforced boundaries**.

Capacity Partition

$$E = M + E_{\text{prop}} \tag{4}$$

Where:

- M = bound capacity
- E_{prop} = propagatable capacity

No additional degrees of freedom are introduced.

Single-Parameter Control All admissible regimes are governed by:

$$f(\varepsilon) = \sqrt{1 - \varepsilon} \tag{5}$$

This reduces system behavior to a **single controlling parameter** without loss of generality.

2.3 Interpretation: Binding vs. Propagation

The identity defines a strict complementarity:

- Binding fraction ε
- Propagation capacity $\sqrt{1 - \varepsilon}$

This implies:

> Propagation is not independent. It is the complement of binding.

There is no regime in which both can increase independently. All system states are constrained by this relation.

2.4 Process and Time (Operational Definition)

Within this framework:

- Time is **not fundamental**
- Time is an **ordering index of state transitions**

A system exhibits process only if:

$$\varepsilon < 1 \tag{6}$$

i.e. only if a propagatable remainder exists.

This removes the need for time as a primitive variable.

2.5 Why This Is Minimal

This identity satisfies:

- **Closure** — no undefined states
- **Irreducibility** — cannot be decomposed further
- **Non-circularity** — no variable defined in terms of itself
- **Falsifiability** — deviations produce measurable contradictions

No additional primitives are required.

2.6 Role in the Framework

This identity is not used for prediction directly.

It serves as:

> the admissibility boundary of all system states

Everything in the framework derives from it:

- projection to observables
- stability evaluation
- hazard computation
- decision gating

2.7 Key Implication

The framework does not evaluate:

> What is likely?

It evaluates:

> What is admissible under this identity?

3. Decision Framework

3.1 From Evaluation to Admissibility

Conventional systems evaluate inputs to produce *best possible outputs*. The presented framework operates differently:

> It evaluates whether an output is **admissible at all**.

No ranking, no approximation, no optimization is performed. A state either satisfies structural conditions—or it does not.

3.2 Deterministic Verdict Space

All evaluations collapse into a finite, closed decision set:

- **ALLOW** — the state is uniquely admissible
- **REFUSE** — the state is underdetermined
- **BLOCK** — the state is structurally invalid

This classification is exhaustive and mutually exclusive.

There are no intermediate confidence values. There is no probabilistic fallback.

3.3 Formal Decision Semantics

The decision is derived from three elements:

1. **Evidence validity**
2. **Invariant satisfaction**
3. **Hazard constraints**

A simplified form of the decision logic is:

- **ALLOW** $\Leftrightarrow$ all invariants satisfied $\wedge$ hazard within bounds
- **REFUSE** $\Leftrightarrow$ missing or insufficient evidence
- **BLOCK** $\Leftrightarrow$ invariant violation

This structure is enforced across domains.

3.4 Fail-Closed Behavior

The framework enforces a strict default:

> If admissibility cannot be proven, execution is denied.

This includes:

- missing measurements
- invalid data
- undefined state variables

In such cases:

- system state = **UNDETERMINED**
- decision = **REFUSE**
- execution = **blocked**

This prevents:

- silent failure
- implicit assumptions
- hidden extrapolation

3.5 Multi-Domain Consistency

The same decision logic applies across all physical domains.

In UPASL, each domain is evaluated independently:

- thermal
- mechanical
- electrical
- radiation
- fluid
- informational

Each domain produces a state:

- SAT (satisfied)
- VIOL (violated)
- UND (undetermined)

Global decision:

- **REFUSE** $\Leftrightarrow$ any domain = VIOL or UND
- **ALLOW** $\Leftrightarrow$ all domains = SAT $\wedge$ constraints satisfied
- **LIMIT** $\Leftrightarrow$ admissible but bounded execution

3.6 Limit as Controlled Admissibility

Unlike binary systems, this framework introduces:

- **LIMIT** — partial admissibility

This allows:

- bounded execution
- controlled degradation
- safe operation under constraints

Example:

- reduced power instead of shutdown
- limited actuation instead of full block

Still:

- fully deterministic
- fully constraint-bound

3.7 Commit-Gated Execution

The decision layer directly controls execution:

> No irreversible action may occur without admissibility.

Examples of commit events:

- power ramp
- actuator engagement
- system mode switch
- deployment events

Execution pipeline:

$$\texttt{State} \rightarrow \texttt{Evaluation} \rightarrow \texttt{Decision} \rightarrow \texttt{Commit Gate} \rightarrow \texttt{Execution}$$

If decision $\neq$ ALLOW, execution is physically prevented.

3.8 No Bypass Condition

A critical property of the system:

> The decision layer is non-bypassable.

- no override
- no manual skip
- no hidden execution path

This is enforced at:

- logic level
- architecture level
- hardware level

3.9 Determinism Guarantee

For identical input:

- same evidence
- same invariants
- same state

the system always produces the same decision.

This ensures:

- reproducibility
- auditability
- verification capability

3.10 Decision = Proof

The central property of the framework:

> A decision is not an outcome. It is a **proof of admissibility**.

- ALLOW — proof exists
- REFUSE — proof incomplete
- BLOCK — contradiction proven

No decision exists without structural justification.

4. Mapping to Observables

4.1 From Internal State to Measurable Reality

A system is only evaluable if its internal state can be mapped to **observable, measurable quantities**.

The presented framework achieves this through a deterministic projection operator:

$$\Pi : \text{State} \rightarrow \text{Observable} \tag{7}$$

This operator transforms internal system variables into:

- measurable physical quantities
- normalized system parameters
- domain-specific metrics

This mapping is not heuristic. It is **structurally defined and reproducible**.

4.2 Bidirectional Structure

A key property of the projection:

> The mapping is bidirectional.

Forward mapping

$$\text{State} \rightarrow \text{Observable} \tag{8}$$

Used for:

- prediction
- system evaluation
- control

Reverse mapping

$$\text{Observable} \rightarrow \text{State} \tag{9}$$

Used for:

- diagnostics
- stability reconstruction
- anomaly detection

This enables complete traceability between theory and measurement.

4.3 Domain Normalization

To ensure cross-domain compatibility, all observables are normalized into a unified range:

$$x_i \in [0, 1] \tag{10}$$

This allows:

- comparison across domains
- aggregation into system-level metrics
- deterministic evaluation independent of units

Example from functional mapping:

- structural integrity
- control stability
- adaptive response

All mapped into normalized axes.

4.4 Multi-Axis Representation

Systems are represented along three functional axes:

- **S (Structure)** — physical stability and integrity
- **M (Modulation)** — control and damping behavior
- **W (Adaptation)** — prediction and response capability

$$F = \{S, M, W\} \tag{11}$$

These axes are derived from measurable quantities and normalized.

4.5 Stability Evaluation

Relative contributions are computed:

$$S_{\text{rel}} = \frac{S}{S + M + W}, \quad M_{\text{rel}} = \frac{M}{S + M + W}, \quad W_{\text{rel}} = \frac{W}{S + M + W} \tag{12}$$

System stability is defined as:

$$\Sigma = 1 - (|S_{\text{rel}} - C_S| + |M_{\text{rel}} - C_M| + |W_{\text{rel}} - C_W|) \tag{13}$$

Interpretation:

- $\Sigma \geq 0.8$ — stable
- $0.5 \leq \Sigma < 0.8$ — borderline
- $\Sigma < 0.5$ — unstable

4.6 Hazard Quantification

Hazards are derived from deviations in each axis:

$$H = \frac{H_S + H_M + H_W}{3} \tag{14}$$

Where each component represents:

- structural deficiency
- control instability
- adaptive failure

This enables:

- explicit risk quantification
- deterministic safety thresholds
- domain-independent hazard detection

4.7 Real-World Measurement Mapping

Each axis is constructed from real measurements.

Structural (S)

- stiffness
- load capacity
- friction
- alignment

Modulation (M)

- damping ratio
- control stability
- latency
- tracking error

Adaptation (W)

- prediction accuracy
- robustness
- adaptation speed

All values are normalized and aggregated.

4.8 Laboratory Workflow (Direct Testability)

The framework defines a concrete testing pipeline:

1. Define system scenario
2. Measure domain-specific parameters
3. Normalize into S, M, W
4. Compute stability and hazard
5. Apply targeted interventions

This enables direct experimental validation without theoretical interpretation.

4.9 Independence from Internal Structure

A critical property:

> Observables can be evaluated without access to internal system logic.

This allows:

- independent verification
- third-party testing
- black-box evaluation

4.10 Key Implication

The system does not rely on:

- hidden variables
- unobservable states
- model-dependent interpretation

Instead:

> Every decision is grounded in measurable reality.

5. Engineering Realization

5.1 From Framework to System Architecture

The presented framework is not an abstract model. It is directly translatable into a layered engineering architecture.

At its core, the system separates:

- state evaluation
- admissibility decision
- execution control

This separation ensures that:

- no execution occurs without validation
- no validation occurs without measurable input
- no decision bypasses structural constraints

5.2 Core System Components

The engineering realization consists of three primary layers.

1. Universal Stability Kernel (USK) The USK is the deterministic core of the system.

It is responsible for:

- evaluating admissibility conditions
- enforcing invariant constraints
- generating deterministic decisions

Key properties:

- fail-closed logic
- no probabilistic branching
- invariant-bound evaluation

2. Universal Physical Admissibility and Stabilisation Layer (UPASL) UPASL extends the kernel into real-world systems.

It integrates multiple domains:

- thermal
- mechanical
- electrical
- radiation
- fluid
- informational

Each domain is evaluated independently and merged into a global decision.

3. Domain-Specific Implementations The framework can be instantiated in concrete systems.

Examples include:

- thermodynamic control systems (TSOL)
- satellite subsystem control with invariant-bound commit gating
- biomechanical stabilization systems using functional axis mapping

5.3 Execution Pipeline

The system operates in a fixed pipeline:

$$\texttt{Measurement} \rightarrow \texttt{Normalization} \rightarrow \texttt{Projection} \rightarrow \texttt{Evaluation} \rightarrow \texttt{Decision} \rightarrow \texttt{Commit Gate} \rightarrow \texttt{Execution}$$

Each step is:

- deterministic
- non-skippable
- structurally bound

No dynamic branching or heuristic shortcuts exist.

5.4 Commit-Gated Control

A central engineering principle:

> Irreversible actions are gated by admissibility.

Examples:

- power activation
- actuator engagement
- transmission ramp
- deployment sequences

If admissibility is not satisfied, execution is physically blocked.

5.5 Hardware Compatibility

The architecture is inherently compatible with hardware implementation.

Reasons:

Deterministic Flow

- no stochastic processes
- no sampling
- no dynamic model updates

Fixed Pipeline

- predictable execution order
- no recursion or uncontrolled loops

Bounded State Space

- all variables constrained
- no undefined states

Boolean Decision Layer

- ALLOW / REFUSE / BLOCK
- directly implementable in logic circuits

This enables:

- FPGA / ASIC realization
- real-time control systems
- safety-critical deployment

5.6 Golden Model Alignment

The system defines a deterministic reference model:

- identical input $\rightarrow$ identical output
- hardware must match reference behavior

This allows:

- formal verification
- hardware validation
- regression testing

5.7 No Heuristic Dependency

Unlike conventional systems:

- no training required
- no dataset dependency
- no parameter tuning through optimization

All behavior is:

- structurally defined
- reproducible
- verifiable

5.8 Integration into Existing Systems

The framework does not replace systems. It integrates as a **control and validation layer**.

Possible integration points:

- pre-execution validation
- safety gating
- system stabilization
- anomaly detection

5.9 Key Engineering Property

> The system enforces correctness before execution.

Not after. Not statistically. Not approximately.

5.10 Summary

The framework provides:

- a deterministic evaluation core (USK)
- a multi-domain integration layer (UPASL)
- direct mapping to real-world systems
- hardware-compatible execution

It transforms admissibility from a conceptual requirement into a **physical constraint of execution**.

6. Falsifiability & Testing

6.1 Principle of Falsifiability

The framework is constructed such that it can be **proven wrong** under clearly defined conditions.

This is not optional.

> If the system produces an admissible decision where invariants are violated, it is falsified. If it refuses or blocks where admissibility is demonstrably satisfied, it is incomplete.

This establishes a strict criterion:

Every decision must be testable against measurable reality.

6.2 Testability Requirements

For a system to be considered evaluable, it must provide:

1. observable inputs
2. deterministic transformation rules
3. reproducible outputs
4. explicit decision criteria

All four are satisfied:

- inputs → measurable parameters
- transformation → projection Π
- evaluation → invariant-bound logic
- output → ALLOW / REFUSE / BLOCK

6.3 Experimental Validation Path

The framework defines a concrete validation pipeline.

Step 1 — Scenario Definition Define a system with known measurable parameters:

- physical system
- control system
- simulated environment

Step 2 — Measurement Acquisition Collect raw observables:

- energy levels
- control variables
- structural parameters

Step 3 — Projection Map measurements into normalized system space:

$$\Pi(\text{measurement}) \to (S, M, W) \tag{15}$$

Step 4 — Evaluation Compute:

- stability index
- hazard values
- invariant satisfaction

Step 5 — Decision System produces:

- ALLOW
- REFUSE
- BLOCK

Step 6 — Outcome Comparison Compare decision with real system behavior:

- stable → ALLOW expected
- unstable → BLOCK expected
- insufficient data → REFUSE expected

6.4 Example: Functional Stability Testing

A direct experimental setup is already defined:

- measure structural, modulation, and adaptive parameters
- normalize into S, M, W
- compute stability and hazard
- apply controlled perturbations

Expected behavior:

- within stability corridor → ALLOW
- outside corridor → BLOCK
- incomplete measurement → REFUSE

6.5 Hypothesis-Driven Testing

The framework generates explicit, testable hypotheses.

Example:

$$0.6 \leq \frac{F}{O} \leq 0.8 \Rightarrow \text{stable system behavior} \tag{16}$$

Deviations from this range must produce:

- measurable instability
- increased variance
- structural degradation

6.6 Simulation-Based Validation

The framework can be tested in simulation environments:

- FEM systems
- control simulations
- physical field models

Procedure:

1. define system parameters
2. vary input conditions
3. track system response
4. compare predicted vs. observed admissibility

6.7 Reverse Validation (Critical Property)

Using reverse mapping:

$$\text{Observable} \rightarrow \text{State} \tag{17}$$

One can:

- reconstruct system state

- detect hidden instability
- identify constraint violations

6.8 Independent Verification

A key design goal:

> The system can be validated without internal access.

Requirements:

- access to inputs
- access to outputs
- access to measurement procedure

Not required:

- internal equations
- implementation details
- proprietary architecture

6.9 Failure Conditions

The system is falsified if:

- decisions are inconsistent for identical inputs
- admissible states are rejected
- invalid states are accepted
- execution occurs without admissibility

6.10 No Interpretive Layer

A critical distinction:

> The framework does not require interpretation to validate results.

Validation is based on:

- measurement
- comparison
- reproducibility

6.11 Key Implication

The framework transforms evaluation into a **testable process** instead of:

- a model assumption
- a statistical approximation
- a subjective interpretation

7. Evaluation Call

7.1 Purpose of This Publication

This document does not aim to establish authority.

It defines a framework that is:

- structurally complete
- operationally defined
- experimentally testable

The purpose of this publication is:

> to invite independent evaluation under real-world conditions.

7.2 Scope of Evaluation

The framework can be evaluated across multiple domains.

Computational Systems

- decision logic validation
- reproducibility under identical inputs
- fail-closed behavior under missing data

Physical Systems

- stability prediction vs. measured behavior
- hazard detection accuracy
- response to controlled perturbations

Engineering Systems

- admissibility-gated execution
- safety constraint enforcement
- deterministic control behavior

7.3 Minimal Evaluation Requirements

Independent evaluation requires only:

1. measurable input parameters
2. a defined projection mapping
3. observable system outcomes

No access is required to:

- internal implementation
- proprietary system layers
- full ontological derivation

7.4 Suggested Test Cases

The following test classes are sufficient to validate core properties.

Test Case 1 — Deterministic Consistency

- identical input → identical decision
- repeated evaluation must yield identical results

Test Case 2 — Boundary Behavior

- approach stability limits
- verify transition from ALLOW → BLOCK

Test Case 3 — Missing Evidence

- remove required measurements
- system must produce REFUSE

Test Case 4 — Constraint Violation

- introduce invariant breach
- system must produce BLOCK

Test Case 5 — Real-World System Mapping

- measure system state
- project into normalized space
- compare predicted vs. actual stability

7.5 Evaluation Criteria

The framework is considered valid if:

- decisions match measurable system behavior
- no admissible state is rejected
- no invalid state is accepted
- all outputs are reproducible

7.6 Open Verification Strategy

The framework is intentionally designed to support:

- black-box testing
- independent replication
- cross-domain validation

This allows:

- academic evaluation
- industrial validation
- engineering integration

without requiring disclosure of core intellectual property.

7.7 What This Framework Does Not Require

- training datasets
- parameter tuning
- probabilistic calibration
- heuristic adjustment

All system behavior is:

- predefined
- deterministic
- invariant-bound

7.8 Expected Outcomes

If the framework holds under evaluation:

- decision-making can be made deterministic
- safety-critical systems can eliminate heuristic risk
- execution can be bound to admissibility instead of probability

If it fails:

- failure conditions will be explicit and reproducible
- contradictions will be measurable
- falsification will be unambiguous

7.9 Final Position

This work introduces a shift in system design:

From:

- generating outputs

To:

- validating admissibility

7.10 Closing Statement

The framework stands or falls on measurable reality.

> It does not ask to be believed. It asks to be tested.

License